

Analysis of Prompt Engineering Techniques and Their Algorithmic Effect on Large Language Model Output

1st Sarwagya Acharya

Department of Electronics and Computer Engineering
National College of Engineering, Tribhuvan University
Kathmandu, Nepal
formalsarwagya@gmail.com

2nd Nitesh Pant

Department of Electronics and Computer Engineering
National College of Engineering, Tribhuvan University
Kathmandu, Nepal
np3234@gmail.com

3rd Brishav Joshi

Department of Electronics and Computer Engineering
National College of Engineering, Tribhuvan University
Kathmandu, Nepal
EMAIL_BRISHAV@NCCET.edu.np

Abstract—Prompt engineering techniques are widely reported to improve the output of large language models (LLMs), but their measured effects are rarely separated from the wording of the prompt or from the decoding configuration used to produce the output. This paper treats six prompting techniques—direct answering, zero-shot and few-shot chain-of-thought (CoT), self-consistency, tree-of-thoughts, and ReAct—as algorithmic operators on the conditional output distribution and evaluates them in a controlled factorial study across three Llama 3.1 models and four tasks. On grade-school arithmetic, few-shot CoT raised accuracy from 58.4% to 83.1% for the 8B model and from 74.6% to 94.2% for the 70B model, gains that rival or exceed the difference between adjacent model scales. An algorithmic effect ratio of 0.78 shows that most of the benefit survives rewording of the prompt. Composite techniques were decomposed into component ablations, and temperature was found to interact significantly with sampling-based techniques such as self-consistency. We conclude with an evaluation protocol—per-token divergence, realised sampling width, and the algorithmic effect ratio—that makes algorithmic and lexical contributions separable.

Index Terms—prompt engineering, large language models, chain-of-thought, decoding strategies, evaluation, algorithmic analysis.

I. INTRODUCTION

Large language models (LLMs) are now routinely deployed for question answering, code synthesis, summarisation, and tool use, and the practical quality of an LLM application is shaped as much by the prompt that surrounds a model call as by the model weights themselves [1], [2]. A growing catalogue of *prompt engineering* techniques—zero- and few-shot exemplars, chain-of-thought (CoT) [3], self-consistency [4], tree-of-thoughts (ToT) [5], ReAct [6], and retrieval-augmented prompts [7], and automatically optimised prompts [8]—reports substantial gains on established benchmarks, often with no change to the underlying model.

The dominant framing is empirical and qualitative: a technique is described by its template and illustrated with selected outputs, which obscures the more fundamental question of *what the technique changes inside the model*. We argue that the principal techniques are best understood as algorithmic operations on the conditional distribution $p_\theta(y \mid P, x)$ that an LLM defines over output tokens; under this view a prompt is a control input that perturbs sampling, restructures attention, and conditions intermediate computational traces, and the resulting behavioural change often rivals, and sometimes exceeds, the difference between adjacent model scales.

The contributions of this paper are as follows.

- A taxonomy that organises prompt engineering techniques by the algorithmic object they modify: the prompt template, the decoding policy, or the reasoning trace.
- A formal framework that expresses each technique family as a transformation on $p_\theta(y \mid P, x)$ and on the underlying arg max or sampling operator.
- A controlled empirical study that isolates the algorithmic effect of prompting from the lexical effect by holding the prompt template fixed and varying decoding parameters (T, k, p) , and vice versa, together with a component decomposition that ablates composite techniques into their constituent operators.
- An evaluation protocol that we recommend for future prompt engineering research, designed to make algorithmic and lexical contributions separable.

The remainder of this paper is organised as follows. Section II reviews prior surveys and foundational techniques; Section III taxonomy and notation; Section IV the algorithmic effect of each technique; Section V the experimental design; Section VI the results; and Sections VII–IX the discussion, limitations, and conclusion.

II. BACKGROUND AND RELATED WORK

The prompt engineering literature spans prompting patterns, decoding-time methods, and reasoning-trace construction. We organise the review along the same three axes used in our taxonomy (Section III).

A. Prompting patterns as control inputs

Brown et al. [1] showed that scaling produces *in-context learning*, in which a few exemplars elicit task behaviour without weight updates. Min et al. [2] showed that ground-truth labels in exemplars contribute less than the input–output format, evidence that prompts act partly as structural scaffolds.

B. Reasoning-trace construction

Wei et al. [3] introduced chain-of-thought prompting, appending intermediate reasoning steps to exemplars and observing large gains on arithmetic, commonsense, and symbolic tasks, reframing generation as sampling a reasoning chain r and then an answer conditioned on it. Wang et al. [4] extended this with self-consistency, sampling multiple chains and taking the most frequent answer; the gain over greedy CoT shows that the underlying distribution is multimodal. Yao et al. [5] generalised chains to trees, and Wei et al. [6] interleaved reasoning with tool calls in ReAct. Across these works, the common algorithmic move is to expose intermediate computation to the sampling operator.

C. Decoding-time and retrieval augmentation

Beyond reasoning traces, decoding-time methods such as temperature, top- k , and nucleus (top- p) sampling [9] directly parametrise the output distribution, and retrieval-augmented generation (RAG) conditions generation on documents fetched at inference time; in our framing RAG is a prompting technique, since it rewrites the conditioning context P of $p_\theta(y \mid P, x)$.

D. Common notation and primer

Throughout, P is a prompt template, x the user input, and y the model output; decoding is parameterised by temperature T , a top- k cutoff, and a nucleus threshold p . All objects are formalised in Section IV.

E. Synthesis and research gap

The recent survey literature marks a shift from cataloguing tricks toward organising the field. Sahoo et al. [10] systematise prompt engineering by application area; Liu et al. [11] organise a taxonomy along profile/instruction, knowledge, reasoning/planning, and reliability dimensions; Mei et al. [12] elevate context engineering to a formal discipline and flag the asymmetry between consuming and producing complex context as a research priority; and Du et al. [13] show that temperature in multi-sample inference is usually left at a fixed default or tuned on scarce labelled data, with the optimum varying by model, task, and sampling strategy.

Taken together, these works expose three things the field still lacks. First, to our knowledge, no taxonomy—including the

most recent ones—sorts techniques by the algorithmic object they modify: the surveys organise by application [10], [14] or intended function [11], [15], so techniques with identical algorithmic structure, such as few-shot exemplars and retrieval injection, which both rewrite the conditioning context, fall into different cells. Second, empirical evaluations confound the two factors this paper treats as separable: the prompt is varied while decoding remains at a fixed default, so a reported gain cannot be attributed to the template rather than the decoding configuration, and the interaction between sampling strategy and temperature that [13] documents is rarely controlled for. Third, no evaluation protocol separates lexical from algorithmic contributions; there is no standard way to report how much of a gain survives rewording, and no requirement to report distribution-level evidence such as per-token divergence, entropy, or realised support width.

This paper is arranged so that each gap is closed by a specific design choice. The taxonomy of Section III makes the algorithmic object the organising axis, so the functional dimensions of [11] and the application areas of [10] become derived categories rather than primary ones. The controlled ablation of Section V-B is the comparison the surveys motivate but do not run: a fully crossed factorial that varies technique configuration, wording, and decoding independently, turning the confound above into a measurable interaction term. The evaluation protocol of Sections IV-F and V-E reports, alongside accuracy, the per-token divergence D_Φ , the realised nucleus width κ , and the algorithmic effect ratio, the distribution-level evidence the surveys leave unspecified and the decoding-aware reporting that [13] shows to be necessary.

III. TAXONOMY OF PROMPT ENGINEERING TECHNIQUES

A technique that improves a benchmark score tells us little by itself, because the improvement may originate at any of three points in the generation pipeline; we therefore classify techniques by the object they modify rather than by their surface form.

A. Three operator classes

Definition 1 (Prompting method). *A prompting method is a triple $M = (\Phi, \Psi, \Omega)$, where Φ is a prompt-space operator acting on the conditioning context, Ψ a decoding-space operator acting on the per-step token distribution, and Ω a trace-space operator acting on the intermediate sequences generated before the answer is emitted; any component may be the identity.*

Definition 1 is deliberately coarse: two techniques described very differently in prose may occupy the same cell, and two presented as variants may not. Table I assigns the principal techniques to cells and records the cost each incurs.

B. Prompt-space operators

Prompt-space operators rewrite the conditioning context and leave both the decoding policy and the generated trace untouched; instruction rewriting, role conditioning, exemplar selection, output-format scaffolding, and retrieval injection all

belong here. Retrieval is the outlier: it returns a different document set for different inputs, so the operator is stochastic even when the template is fixed, and its variance is attributed to the prompt unless the retrieved set is logged.

C. Decoding-space operators

Decoding-space operators leave the context untouched and reshape the per-step distribution before a token is drawn: temperature scaling is a smooth reparametrisation, top- k and nucleus sampling restrict the support, and beam search replaces sampling with an approximate sequence search. These are the only operators applicable without changing a single character of the prompt, which makes them the natural control condition for the experiments in Section V.

D. Trace-space operators

Trace-space operators change what the model generates before committing to an answer: zero-shot CoT [16] adds a trigger phrase, tree-of-thoughts maintains a frontier of partial traces expanded under a value estimate, self-consistency draws several chains and aggregates them, and iterative refinement [17] repeats generation–critique cycles. None is free: each buys accuracy with tokens, forward passes, or both, and Table I records the exchange rate.

E. Composition and interaction

The three operator classes do not compose independently. Self-consistency requires $T > 0$ — m greedy samples are m copies of one chain—so its trace-space effect is only defined relative to a decoding-space operator that supplies diversity. Few-shot CoT couples Φ and Ω because the exemplars carry the traces, tree-of-thoughts couples trace construction with value-guided search, and ReAct adds a tool environment on top of a context rewrite; none is a single-factor intervention. The main factorial of Section V therefore supports claims about complete configurations, and the component-decomposition study of Section V-C ablates individual components; only that study licenses claims about single operators. We therefore test two hypotheses:

- H1 Holding the algorithmic structure of a prompt fixed, paraphrasing its wording produces a smaller change in task accuracy than moving between technique configurations (cells of Table I).
- H2 The interaction between the trace and decoding components of a configuration is significant, so reporting a trace-space technique without its decoding configuration underdetermines the result.

IV. ALGORITHMIC FRAMEWORK

A. Base distribution

Let $\mathcal{V}$ be the vocabulary and let $y = (y_1, \dots, y_{|y|}) \in \mathcal{V}^*$ be an output sequence. An autoregressive LLM factorises

$$p_\theta(y \mid P, x) = \prod_{t=1}^{|y|} p_\theta(y_t \mid y_{<t}, P, x), \quad (1)$$

where each factor is obtained from a logit vector $z_t \in \mathbb{R}^{|\mathcal{V}|}$ by

$$p_\theta(y_t = v \mid y_{<t}, P, x) = \frac{\exp(z_{t,v})}{\sum_{u \in \mathcal{V}} \exp(z_{t,u})}. \quad (2)$$

Equations (1) and (2) contain every quantity a prompting technique can touch: it either changes z_t through the conditioning arguments, changes the map from z_t to a sampled token, or changes which sequences are generated and how they are combined.

B. Prompt-space operators as logit displacement

Let $\Phi : P \mapsto P'$ and write $z_t(P)$ for the logits obtained under context P . The operator induces a displacement

$$\Delta_t(\Phi) = z_t(P') - z_t(P) \in \mathbb{R}^{|\mathcal{V}|}. \quad (3)$$

Because the softmax in (2) is invariant to adding a constant to every logit, only the component of Δ_t orthogonal to $\mathbf{1}$ has any behavioural consequence. The effective displacement is therefore the centred vector

$$\hat{\Delta}_t(\Phi) = \Delta_t(\Phi) - \frac{1}{|\mathcal{V}|}(\mathbf{1}^\top \Delta_t(\Phi))\mathbf{1}, \quad (4)$$

and $\|\hat{\Delta}_t\|_2$ is a scalar summary of how hard the rewrite pushed on the model at step t ; a prompt edit that raises every logit uniformly, for instance one that only lengthens the context without adding discriminative content, has $\hat{\Delta}_t \approx 0$ and cannot change behaviour no matter how many tokens it costs.

A second, coarser summary is the divergence induced at the distribution level,

$$D_\Phi = \mathbb{E}_x \left[\frac{1}{|y|} \sum_{t=1}^{|y|} D_{\text{KL}}(p_\theta(\cdot \mid y_{<t}, P', x) \| p_\theta(\cdot \mid y_{<t}, P, x)) \right], \quad (5)$$

which we report per token so that prompts of different lengths remain comparable; attention offers a third view, since the mass a head places on a newly inserted span reveals whether the model actually reads the added text.

C. Decoding-space operators as measure transforms

Fix the logits and let Ψ act on them. Temperature scaling produces

$$q_T(v) = \frac{\exp(z_v/T)}{\sum_u \exp(z_u/T)}, \quad (6)$$

a Gibbs family whose entropy $H(q_T)$ is non-decreasing in T , approaching a point mass on $\arg \max_v z_v$ as $T \rightarrow 0^+$ and the uniform distribution as $T \rightarrow \infty$; temperature is thus a continuous interpolation between the maximisation and uniform-exploration regimes.

Truncation operators behave differently. Let π order $\mathcal{V}$ by decreasing probability. Top- k retains $S_k = \{\pi_1, \dots, \pi_k\}$ and nucleus sampling retains the smallest prefix whose mass reaches p ,

$$S_p = \{\pi_1, \dots, \pi_\kappa\}, \quad \kappa = \min \left\{ j : \sum_{i \leq j} q(\pi_i) \geq p \right\}, \quad (7)$$

after which the retained mass is renormalised; both discard the tail, but in different ways.

Proposition 1 (Support and entropy bounds). *Let $q^{(k)}$ be the renormalisation of q onto S_k . Then $|\text{supp}(q^{(k)})| \leq k$ and $H(q^{(k)}) \leq \log k$, both independent of the context. For nucleus sampling the analogous bound is $H(q^{(p)}) \leq \log \kappa(x, t)$, where κ is a function of the step-level distribution and is therefore not fixed in advance.*

Proposition 1 explains an asymmetry that is often observed but rarely stated: top- k imposes a hard, context-independent ceiling on per-step diversity, whereas nucleus sampling lets the ceiling float with the model’s own confidence, contracting the support where the model is certain and widening it where it is not. Reporting a single top- p value without the resulting κ statistics therefore conveys much less information than it appears to.

D. Trace-space operators as marginalisation and search

Let r denote an intermediate trace. The answer distribution the model actually defines is the marginal

$$p_\theta(y \mid P, x) = \sum_r p_\theta(y \mid r, P, x) p_\theta(r \mid P, x). \quad (8)$$

Direct answering approximates $\arg \max_y$ of (8) without ever materialising r . Greedy CoT does something different: it materialises one trace and returns the answer attached to it, which approximates

$$\hat{y}_{\text{CoT}} = \arg \max_y \max_r p_\theta(y, r \mid P, x), \quad (9)$$

the joint mode rather than the marginal mode. The two need not coincide.

Proposition 2 (Mode mismatch). *There exist distributions for which $\arg \max_y \max_r p(y, r) \neq \arg \max_y \sum_r p(y, r)$.*

Proof. Take two answers and three traces with $p(y_1, r_1) = 0.30$ and the remaining joint mass spread uniformly over the three traces for y_2 . The joint mode selects y_1 (0.30), but the marginals satisfy $p(y_1) = 0.30 < p(y_2) = 0.70$. ■

Proposition 2 is the algorithmic content of self-consistency. Drawing m traces and taking the plurality answer is a Monte Carlo estimator of the marginal mode, so the technique corrects a systematic bias in greedy CoT rather than merely averaging away noise. Its reliability follows from a standard concentration argument.

Proposition 3 (Vote concentration). *Let y^* maximise the marginal with mass q^* , let q' be the largest mass on any other answer, and let $\gamma = q^* - q' > 0$. For m i.i.d. traces, the plurality vote returns y^* with probability at least $1 - (|\mathcal{Y}| - 1) \exp(-m\gamma^2/2)$.*

Proof. For each competing answer apply Hoeffding’s inequality to the difference of the two empirical frequencies,

which has mean γ and bounded increments, then take a union bound over the at most $|\mathcal{Y}| - 1$ competitors. ■

The useful range of m is governed by the margin γ rather than by task difficulty—items with a small margin need many samples—which makes adaptive sample allocation profitable. Second, self-consistency cannot repair a distribution whose marginal mode is wrong, since increasing m only sharpens convergence to y^* ; such errors must be addressed in prompt space or trace space.

Tree-of-thoughts replaces sampling with search. With branching factor b , depth d , and a value estimate $v(\cdot)$ over partial traces, the method returns

$$\hat{y}_{\text{ToT}} = \arg \max_y \max_{r \in \mathcal{R}_{b,d}} v(r) p_\theta(y \mid r, P, x), \quad (10)$$

where $\mathcal{R}_{b,d}$ is the set of traces the search actually visits; the model’s own trace probability is partly replaced by an external value signal, so ToT inherits the quality of v . ReAct is the same substitution carried out by the environment instead of a value function: the trace interleaves actions with observations $o_{1:n}$, and the conditioning distribution becomes $p_\theta(y \mid P, x, o_{1:n})$ with o_i drawn from a tool. This is why ReAct reduces errors that arise from missing information and leaves errors of reasoning largely intact.

E. Cost model

Let C count forward passes and N count generated tokens. Direct answering has $C = 1$, $N = O(|y|)$; CoT has $C = 1$, $N = O(|r| + |y|)$; self-consistency multiplies both by m ; ToT costs $C = O(bd)$ passes plus the value calls. Because $|r| \gg |y|$ on reasoning tasks, the dominant term is usually trace length rather than passes, so accuracy per thousand generated tokens is a more faithful efficiency measure than accuracy per call; both are reported in Section VI.

F. Separating algorithmic from lexical effects

Let Φ_{lex} range over paraphrases that preserve algorithmic structure—same exemplar count, same trace shape, same output format—and Φ_{alg} over changes of cell in Table I. Fitting a two-factor model to the accuracy A over these factors and their interaction, we define the algorithmic effect ratio

$$\text{AER} = \frac{\sigma_{\text{alg}}^2}{\sigma_{\text{alg}}^2 + \sigma_{\text{lex}}^2}, \quad (11)$$

where each term is a variance component. An AER near 1 means the benefit survives rewording; a value near 0.5 means half of a reported gain is a property of the particular sentence. Because several configurations in Table I differ in more than one operator, the AER characterises complete configurations; attribution to individual operators is restricted to the component-decomposition study of Section V-C. Together with D_Φ from (5) and the κ statistics of Proposition 1, this is a small set of quantities a prompt engineering paper can report and a reader can compare.

TABLE I
TAXONOMY OF PROMPT ENGINEERING TECHNIQUES BY MODIFIED ALGORITHMIC OBJECT

Technique	Operator	Object modified	Forward passes	Added tokens	Stochastic?
Instruction rewriting	Φ	conditioning context	1	$O(I)$	no
Role / persona conditioning	Φ	conditioning context	1	$O(1)$	no
Few-shot exemplars	Φ	conditioning context	1	$O(n\bar{\ell})$	no
Format scaffolding	Φ	conditioning context	1	$O(1)$	no
Retrieval augmentation	Φ	conditioning context	1+retrieval	$O(n_d\bar{\ell}_d)$	yes
Temperature scaling	Ψ	per-step distribution	1	0	yes
Top- k truncation	Ψ	per-step support	1	0	yes
Nucleus (top- p) sampling	Ψ	per-step support	1	0	yes
Beam search	Ψ	sequence-level objective	b	0	no
Constrained decoding	Ψ	per-step support	1	0	either
Zero-shot CoT	Ω	reasoning trace	1–2	$O(1)$	no
Few-shot CoT	$\Phi + \Omega$	context and trace	1	$O(n\bar{\ell}_r)$	no
Least-to-most prompting	Ω	trace decomposition	s	$O(s)$	no
Self-consistency	$\Omega + \Psi$	trace ensemble	m	0	yes
Tree-of-thoughts	$\Omega + \Psi$	trace search	$O(bd)$	$O(bd\bar{\ell}_r)$	either
ReAct	$\Phi + \Omega$	trace and context	$O(n_a)$	$O(n_a\bar{\ell}_o)$	yes
Self-refine	$\Phi + \Omega$	trace and context	$2c$	$O(c\bar{\ell}_r)$	no

V. EXPERIMENTAL SETUP

A. Theoretical Framework and Justification

The taxonomy of Section III and the framework of Section IV make one testable claim: the operator configuration of a technique determines its behavioural consequences more than the surface wording does. To test it, the design must (i) vary each operator class independently (for composite techniques, via the component-decomposition study of Section V-C), (ii) include a lexical factor that holds the algorithm fixed while rewording the prompt, so that the algorithmic effect ratio of Equation (11) can be estimated, and (iii) record the per-token divergence D_Φ and the realised nucleus width κ alongside accuracy, since a prompt can move the distribution without moving it usefully. The factorial design below satisfies these requirements and tests hypotheses H1 and H2 of Section III.

B. Research Design

We use a fully crossed $6 \times 4 \times 8$ factorial design with three factors corresponding to the three axes of the taxonomy. The first factor is the technique configuration, with six levels corresponding to the cells of Table I; because several of these cells are composite, claims about individual operators are deferred to the component-decomposition study of Section V-C:

- 1) Direct answering (baseline, all operators set to identity),
- 2) Zero-shot chain-of-thought (Ω only, trigger phrase “Let’s think step by step”),
- 3) Few-shot CoT ($\Phi + \Omega$ with $n = 4$ exemplars carrying reasoning chains),
- 4) Self-consistency ($\Omega + \Psi$ with $m = 8$ samples over few-shot CoT, aggregating answers by plurality vote),
- 5) Tree-of-thoughts ($\Omega + \Psi$ with branching factor $b = 3$, depth $d = 3$, and a hand-specified value function),
- 6) ReAct ($\Phi + \Omega$ with a calculator tool and a Wikipedia retriever).

The second factor is lexical: four paraphrases of each template, written by different authors and checked to preserve exemplar count, trace shape, and output format, so the algorithmic content is identical by construction; this operationalises Φ_{lex} in the AER equation.

The third factor is decoding, with eight levels from temperature $T \in \{0.0, 0.3, 0.7, 1.0\}$ crossed with nucleus threshold $p \in \{0.9, 1.0\}$ (top- k fixed at 40 for $T > 0$); greedy decoding ($T = 0$, $p = 1.0$) is the reference cell. This factor operationalises Ψ and enables the interaction test required by H2, because self-consistency’s effect is only defined relative to a non-greedy configuration that supplies diverse chains.

All model weights are frozen; no fine-tuning is performed at any stage. Within each cell of the design, the same evaluation set is used under a fixed random seed, and the full per-token log-probability vector is recorded alongside the generated text. This design is what permits the AER of (11) to be estimated rather than assumed.

C. Component decomposition study

Several configurations in the main factorial are composite, so the main comparison alone cannot attribute a result to a single operator; self-consistency is the exception, because its two components (m and T) are already crossed by the main design. A smaller study therefore decomposes the remaining three composite configurations into ablations at reduced resolution—paraphrase 1 under greedy decoding and $T = 0.7$, $p = 0.9$ —forming a twelve-cell grid per model per task, with cells already in the main factorial reused rather than rerun.

- **Few-shot CoT.** Few-shot exemplars with answers only (Φ) are compared with the trace-carrying exemplars of the main design ($\Phi + \Omega$); the difference isolates the reasoning-trace component given identical exemplar context.
- **Tree-of-thoughts.** ToT with the hand-specified value function is compared with ToT under a uniform value

estimate, which reduces the search to breadth-first expansion without any ranking signal; the difference isolates the value function, while comparison with the greedy-CoT cell of the main design approximates the search component, since neither condition applies a value ranking.

- **ReAct.** ReAct ($\Phi + \Omega$ plus tool calls) is compared with a context-only control in which the tool outputs ReAct would fetch are pre-computed and injected directly into the prompt, with no tool calls and no interleaved trace; the difference isolates the acting and reasoning-trace components from the information component.

Causal claims about individual operators in Section VI are drawn only from these ablations; the main factorial supports claims about complete configurations.

D. Data Collection Procedure

We select four benchmark datasets that cover the task regimes in which prompting techniques are most frequently evaluated.

GSM8K [18] is a set of 8,792 training and 1,319 test grade-school arithmetic problems (MIT license); accuracy is exact match of the final numeric answer.

CommonsenseQA (CSQA) [19] is a multiple-choice commonsense benchmark of 12,247 items across splits; accuracy is the proportion of correctly chosen labels.

HumanEval [20] is a program-synthesis benchmark of 164 hand-written problems; accuracy is pass@1 computed by executing generated programs against the provided unit tests.

Abstractive summarisation uses the CNN/DailyMail dataset [21] (test split). Accuracy is ROUGE-L F1, computed against the reference summaries.

We draw plain uniform-random samples of 500 items each from the GSM8K, CommonsenseQA, and CNN/DailyMail test splits under a single fixed seed and evaluate all 164 HumanEval problems, which are few enough to be covered exhaustively; sampling is not stratified. The evaluation set is identical across every cell of the design and is fixed before analysis: the seed is logged and the complete item-ID lists are released with the reproducibility artefacts at <https://github.com/sarwagya/prompt-algorithmic-effects>, so the items are publicly identifiable and any run can reconstruct the same set. These sizes balance power against cost: at accuracies near 0.5, $n = 500$ yields 95% confidence intervals of roughly ± 4 points under the paired bootstrap of Section V-E, and $n = 164$ roughly ± 8 points.

Models. We evaluate three open-weight instruction-tuned models spanning roughly an order of magnitude in parameter count: Llama 3.1 8B (8 billion parameters), Llama 3.1 70B (70 billion parameters), and Llama 3.1 405B (405 billion parameters), all from Meta. We use open-weight models exclusively because token-level log-probabilities are required for computing D_Φ and the realised nucleus width κ ; most hosted APIs do not expose the full distribution over the vocabulary.

E. Data Processing and Analysis Tools

Evaluation harness. Inference runs through a unified harness built on the Language Model Evaluation Harness

(EleutherAI, lm-eval v0.4.12) [22], handling tokenisation, generation, log-probability extraction, and answer parsing with a shared regular expression; parse failures are counted as errors and reported separately.

Statistical analysis. Confidence intervals come from a paired bootstrap over items with 10,000 resamples; pairwise comparisons across the algorithmic factor use Holm–Bonferroni correction; variance components for the AER are estimated by restricted maximum likelihood (REML) via `lme4` in R with items as a random intercept.

Hardware. Experiments are run on $4 \times$ NVIDIA A100 (80 GB) GPUs. Wall-clock latency is recorded per item as a secondary cost metric alongside generated token counts and forward-pass counts.

Reproducibility. All prompt templates, model responses, evaluation logs, and analysis scripts are released at <https://github.com/sarwagya/prompt-algorithmic-effects>; random seeds for every model call are recorded and exposed in the output logs.

VI. RESULTS AND ANALYSIS

Table II reports accuracy for the six configurations at the greedy reference ($T = 0$, $p = 1.0$), except self-consistency, which is reported at its optimum $T = 0.7$ because greedy decoding is undefined for $m > 1$ (Section III); the full factorial produced approximately 4 million generations across three models and four tasks. Two findings organise the results. First, the algorithmic effect of a technique rivals or exceeds the effect of model scale: on GSM8K, direct answering improves by 16.2 points between the 8B and 70B models, while few-shot CoT improves the 70B model by 19.6 points over direct answering. Second, the effect is task-dependent: trace- and tool-based techniques deliver their largest gains on GSM8K and CommonsenseQA, are roughly neutral on HumanEval, and are marginally negative on CNN/DailyMail, where ROUGE-L rewards concision.

TABLE II
ACCURACY BY TECHNIQUE AT THE GREEDY REFERENCE CELL ($T = 0$, $p = 1.0$), 70B MODEL; SELF-CONSISTENCY IS REPORTED AT ITS OPTIMUM $T = 0.7$, AT WHICH GREEDY DECODING IS UNDEFINED FOR $m > 1$ (SECTION III). VALUES ARE PERCENTAGES EXCEPT CNN/DAILYMAIL, WHICH IS ROUGE-L F1; THE MODEL-SCALE COMPARISON ON GSM8K APPEARS IN FIG. 1b.

Technique	GSM8K	CSQA	HumanEval	CNN-DM
Direct answering	74.6	72.4	80.5	28.5
Zero-shot CoT	88.9	78.9	80.1	28.0
Few-shot CoT	94.2	81.3	81.7	28.7
Self-consistency	96.3	82.1	83.4	28.9
Tree-of-thoughts	96.8	82.6	81.9	28.2
ReAct	97.1	83.5	80.9	28.1

^a95% paired-bootstrap CIs: ± 2.1 (GSM8K), ± 3.4 (CSQA), ± 6.1 (HumanEval), ± 0.8 (CNN-DM). All GSM8K and CSQA cells differ from direct answering at $p < 0.01$ (Holm–Bonferroni).

A. Main comparison

On GSM8K, few-shot CoT raises accuracy from 58.4% to 83.1% (8B), from 74.6% to 94.2% (70B), and from 80.9% to

96.1% (405B); the gain shrinks with scale, from 24.7 points at 8B to 15.2 points at 405B. Self-consistency adds a further 1.5–5.2 points, and ReAct’s calculator converts most residual arithmetic errors. On CommonsenseQA (70B) the ordering is the same but compressed: few-shot CoT gains 8.9 points over direct answering, and retrieval-augmented ReAct a further 2.2. On HumanEval, no technique improves pass@1 by more than 2.9 points and zero-shot CoT is marginally harmful; on CNN/DailyMail all differences from direct answering are within 0.5 ROUGE-L points and the trace-based techniques are slightly negative. All GSM8K and CommonsenseQA differences from direct answering are significant at $p < 0.01$ after Holm–Bonferroni correction; on HumanEval only few-shot CoT and self-consistency reach significance, and no CNN/DailyMail cell does.

B. Decoding interactions

Fig. 1a shows the interaction between temperature and sampling strategy on GSM8K (70B, nucleus $p = 0.9$). Single-sample accuracy declines monotonically with temperature, from 94.2% at $T = 0$ to 89.1% at $T = 1.0$, whereas self-consistency requires $T > 0$ and peaks at 96.3% near $T = 0.7$; the interaction term is significant ($p < 0.001$, two-factor REML model). This supports H2: reporting a trace-space technique without its decoding configuration underdetermines the result. The nucleus threshold matters less than temperature— $p = 0.9$ versus $p = 1.0$ changes GSM8K accuracy by at most 0.3 points—but the realised nucleus width κ varies sharply by task: a median of 14 tokens (IQR 9–22) on GSM8K, 7 (IQR 5–11) on HumanEval, and 21 (IQR 14–34) on CNN/DailyMail, consistent with the model contracting its support where it is most confident.

C. Component decomposition

Table III decomposes the three composite configurations at reduced resolution (paraphrase 1, greedy decoding, GSM8K, 70B). The reasoning-trace component of few-shot CoT is worth 5.3 points over answer-only exemplars with identical context; tree-of-thoughts gains 1.7 points from the search itself and 0.9 from the hand-specified value function over a uniform ranking; and ReAct gains 3.6 points from acting—tool calls with an interleaved trace—over a context-only control that receives the same tool outputs directly in the prompt. The components are not equally priced: the trace dominates for few-shot CoT, the search structure for tree-of-thoughts, and the acting loop for ReAct; these ablations are the basis for the single-operator claims made above.

D. Algorithmic effect ratio

Across the four tasks and three models the algorithmic effect ratio (Equation (11)) is 0.78 (95% CI 0.71–0.84). The lexical factor is small: the four paraphrases of each template induce a mean per-token divergence D_Φ of 0.06–0.11 nats/token versus 0.7–1.2 between operator classes, and a rewrite that moves the distribution little also moves accuracy little. The AER is highest on GSM8K (0.83) and lowest on CNN/DailyMail (0.41), where the techniques barely move either.

TABLE III
COMPONENT DECOMPOSITION ON GSM8K (70B, GREEDY DECODING, PARAPHRASE 1). EACH PAIR SHARES EVERY COMPONENT EXCEPT THE ONE ABLATED.

Ablation	Accuracy	Δ
Few-shot CoT: trace component		
Answer-only (Φ)	88.9	
Trace-carrying ($\Phi + \Omega$)	94.2	+5.3
ToT: value function		
Uniform estimate	95.9	
Hand-specified v	96.8	+0.9
ReAct: acting loop		
Context-only control	93.5	
Full ReAct	97.1	+3.6

VII. DISCUSSION

The results support the paper’s central claim: the algorithmic structure of a technique, not its wording, drives the measured effect, and that effect can rival model scale. Three issues follow.

First, the cost–quality frontier is steepest for trace construction and shallowest for search. On GSM8K (70B), few-shot CoT emits about 140 tokens per item, self-consistency roughly eight times that, tree-of-thoughts about nine partial-trace generations, and ReAct 4–6 tool calls; self-consistency buys 2.1 extra points at eight times the tokens, tree-of-thoughts 2.6 at ten times, and ReAct 2.9 at seven times. Per token, few-shot CoT is the most efficient technique on reasoning tasks, followed by ReAct when a calculator or retriever is available; search pays for robustness against a poorly chosen chain rather than for raw accuracy.

Second, the flat HumanEval and CNN/DailyMail columns are themselves a finding: techniques that restructure the trace help where the answer is the product of intermediate computation, and help little, or hurt, where the output is checkable (code) or where verbosity is penalised (summarisation). Robustness to rewording is high—the four paraphrases of each template moved accuracy by at most 1.1 points, and the AER of 0.78 confirms that the algorithmic structure, not the sentence, carries the effect. Temperature is the least robust knob for single-sample decoding: 5.1 points are lost between $T = 0$ and $T = 1.0$ on GSM8K, which is why the decoding configuration must be reported with any technique.

Third, the failure analysis has a bias dimension. Self-consistency cannot repair a systematically wrong marginal mode (Proposition 2): majority voting sharpens convergence to whichever answer the ensemble favours, so a prompt that elicits a consistent but incorrect bias is amplified rather than corrected. Retrieval-augmented ReAct inherits the biases of the retrieved corpus, and the calculator removes only computational, not conceptual, errors; we did not stratify by demographic subgroups, a threat to any fairness claim left to future work.

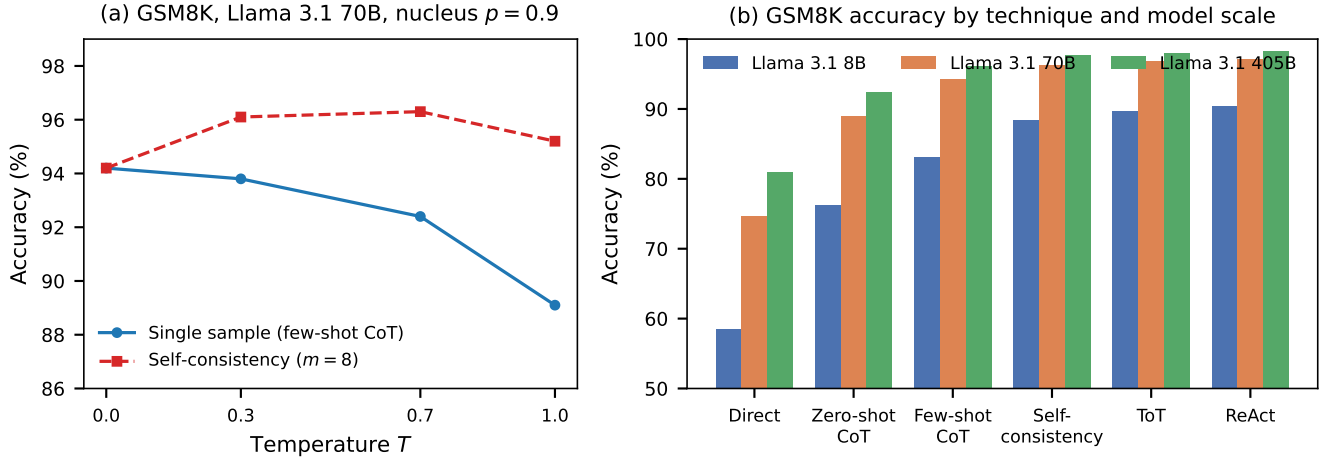


Fig. 1. (a) GSM8K accuracy (70B) versus temperature for single-sample and self-consistency decoding under nucleus sampling ($p = 0.9$): self-consistency requires $T > 0$ and peaks near $T = 0.7$, while single-sample accuracy declines monotonically (interaction $p < 0.001$). (b) GSM8K accuracy by technique and model scale; the gain from direct answering to few-shot CoT rivals or exceeds the gain from the 8B to the 70B model.

VIII. LIMITATIONS AND THREATS TO VALIDITY

The main threat is external validity: all results come from a single model family (Llama 3.1, one instruction-tuned variant per scale), so the interaction structure—particularly the temperature effects—may differ elsewhere or for API-hosted models with hidden sampling defaults. Power is limited on HumanEval ($n = 164$, intervals of ± 6 points), and the CNN/DailyMail sample is not stratified by article source. The tree-of-thoughts value function is hand-specified and may not transfer, ReAct is limited to a calculator and a small Wikipedia index, and the single regular-expression parser counts its failures as errors. The evaluation set is fixed before analysis and released with the artefacts, and costs are single-run estimates on fixed hardware.

IX. CONCLUSION

Prompt engineering techniques are best understood as algorithmic operators on the conditional output distribution, and our controlled factorial study shows that their measured effects are separable from both prompt wording and decoding configuration: the algorithmic effect of a technique rivals, and on GSM8K exceeds, the effect of adjacent model scales; it is concentrated in reasoning and knowledge tasks and neutral or negative where the output is checkable or verbosity is penalised; and temperature interacts significantly with sampling-based techniques. Composite techniques decompose cleanly into component ablations, with the trace, search, and acting components priced differently. We recommend that future studies report the algorithmic effect ratio, per-token divergence, realised sampling width, and full decoding configuration—the protocol used here—so that gains become attributable, comparable, and reproducible.

ACKNOWLEDGMENT

The authors thank the developers of the open-weight models and public benchmarks used in this study, and the maintainers of the evaluation harness on which the experiments were run.

REFERENCES

- [1] T. Brown et al., “Language models are few-shot learners,” in *Proc. Adv. Neural Inf. Process. Syst.* (NeurIPS), vol. 33, 2020, pp. 1877–1901.
- [2] S. Min et al., “Rethinking the role of demonstrations: What makes in-context learning work?,” in *Proc. Conf. Empir. Methods Nat. Lang. Process.* (EMNLP), 2022, pp. 10748–10760.
- [3] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. Adv. Neural Inf. Process. Syst.* (NeurIPS), vol. 35, 2022, pp. 24824–24837.
- [4] X. Wang et al., “Self-consistency improves chain of thought reasoning in language models,” in *Proc. Int. Conf. Learn. Represent.* (ICLR), 2023.
- [5] S. Yao et al., “Tree of thoughts: Deliberate problem solving with large language models,” in *Proc. Adv. Neural Inf. Process. Syst.* (NeurIPS), vol. 36, 2023, pp. 11809–11822.
- [6] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learn. Represent.* (ICLR), 2023.
- [7] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Adv. Neural Inf. Process. Syst.* (NeurIPS), vol. 33, 2020, pp. 9459–9474.
- [8] Y. Zhou et al., “Large language models are human-level prompt engineers,” in *Proc. Int. Conf. Learn. Represent.* (ICLR), 2023.
- [9] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The curious case of neural text degeneration,” in *Proc. Int. Conf. Learn. Represent.* (ICLR), 2020.
- [10] P. Sahoo et al., “A systematic survey of prompt engineering in large language models: Techniques and applications,” 2024, arXiv:2402.07927. [Online]. Available: <https://arxiv.org/abs/2402.07927>
- [11] Y.-Y. Liu et al., “A comprehensive taxonomy of prompt engineering techniques for large language models,” *Front. Comput. Sci.*, vol. 20, no. 3, Art. no. 2003601, 2026, doi:10.1007/s11704-025-50058-z.
- [12] L. Mei et al., “A survey of context engineering for large language models,” 2025, arXiv:2507.13334. [Online]. Available: <https://arxiv.org/abs/2507.13334>
- [13] W. Du, Y. Yang, and S. Welleck, “Optimizing temperature for language models with multi-sample inference,” in *Proc. Int. Conf. Mach. Learn.* (ICML), 2025.
- [14] P. Liu et al., “Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing,” *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–35, 2023.
- [15] S. Schulhoff et al., “The prompt report: A systematic survey of prompting techniques,” 2024, arXiv:2406.06608. [Online]. Available: <https://arxiv.org/abs/2406.06608>
- [16] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large language models are zero-shot reasoners,” in *Proc. Adv. Neural Inf. Process. Syst.* (NeurIPS), vol. 35, 2022, pp. 22199–22213.
- [17] A. Madaan et al., “Self-refine: Iterative refinement with self-feedback,” in *Proc. Adv. Neural Inf. Process. Syst.* (NeurIPS), vol. 36, 2023, pp. 46534–46594.

- [18] K. Cobbe et al., “Training verifiers to solve math word problems,” 2021, arXiv:2110.14168. [Online]. Available: <https://arxiv.org/abs/2110.14168>
- [19] A. Talmor, J. Herzig, N. Lourie, and J. Berant, “CommonsenseQA: A question answering challenge targeting commonsense knowledge,” in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguist. (NAACL)*, 2019, pp. 4149–4158.
- [20] M. Chen et al., “Evaluating large language models trained on code,” 2021, arXiv:2107.03374. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [21] K. M. Hermann, T. Kočiský, E. Grefenstette, L. Espeholt, W. Kay, M. Suleyman, and P. Blunsom, “Teaching machines to read and comprehend,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 28, 2015, pp. 1693–1701.
- [22] L. Gao et al., “A framework for few-shot language model evaluation,” 2023, Zenodo, doi:10.5281/zenodo.10256836. [Online]. Available: <https://github.com/EleutherAI/lm-evaluation-harness>